```
import java.util.*;
public class WaterJug {
   public static void main(String[] args) {
      Stack<int[]> st = new Stack<>();
      Set<String> seen = new HashSet<>();
      st.push(new int[]{0, 0});
      while (!st.empty()) {
         int[] s = st.pop();
         int a = s[0], b = s[1];
         if (!seen.add(a + "," + b)) continue;
         System.out.println("4L Jug = " + a + ", 3L Jug = " + b);
         if (a == 2 || b == 2) {
            System.out.println("Goal Reached!");
            break;
         }

         st.push(new int[]{4, b});
         st.push(new int[]{a, 3});
         st.push(new int[]{0, b});
         st.push(new int[]{a, 0});
         st.push(new int[]{a - Math.min(a, 3-b), b + Math.min(a, 3-b)});
         st.push(new int[]{a + Math.min(b, 4-a), b - Math.min(b, 4-a)});
      }
   }
}
```

1. Theory

Depth First Search (DFS) is an uninformed search technique used to solve problems.

- DFS explores one branch as deeply as possible.
- If it reaches a dead end, it goes back and tries another path. This is called backtracking.
- DFS uses a Stack (LIFO – Last In, First Out) or recursion.
- It uses a visited set to avoid repeating states.
- DFS does not always find the shortest path.

Water Jug Problem:

- Two jugs: 4 litres and 3 litres.
- Initial state: (0, 0)
- Goal state: (2, 0) or (0, 2)
- Actions: Fill, empty, and pour water between jugs.

Here, (x, y) represents water in the 4L and 3L jugs.

2. Algorithm

1. Start.
2. Define initial state (0,0) and goal state (2,0) or (0,2).
3. Create an empty stack and visited set.
4. Push the initial state into the stack.
5. Pop the top state from the stack.
6. Check whether the goal is reached.
7. If not, generate all valid next states.
8. Push unvisited states into the stack.
9. If a dead end occurs, backtrack.
10. Repeat until the goal is reached or the stack becomes empty.
11. Display the solution path.

12. Stop.

3. Viva Questions and Answers

Q1. What is DFS?

DFS is a search technique that explores one branch deeply before trying another.

Q2. Is DFS informed or uninformed?

Uninformed search.

Q3. Which data structure is used in DFS?

Stack (LIFO).

Q4. What is LIFO?

Last In, First Out.

Q5. What is BFS?

Breadth First Search explores states level by level using a Queue.

Q6. Difference between DFS and BFS?

DFS uses Stack; BFS uses Queue.

Q7. What is backtracking?

Going back when no further valid move is possible and trying another path.

Q8. Does DFS always find the shortest path?

No.

Q9. Why is a visited set used?

To avoid visiting the same state repeatedly.

Q10. What is the initial state in the Water Jug Problem?

(0, 0).

Q11. What is the goal state?

(2, 0) or (0, 2).

Q12. What are the operations in the Water Jug Problem?

Fill, empty, and pour water.

Q13. Give applications of DFS.

Maze solving, Water Jug Problem, and path finding.

Q14. What are the limitations of DFS?

It may not find the shortest path and can explore unnecessary deep paths.

Q15. What is a state in the Water Jug Problem?

The amount of water present in both jugs.